



MASTERS IN MECHANICAL ENGINEERING

ROBOTIC MANIPULATION

Laboratory Project: Mako Surgical Robot

1st Part: Robot Kinematics

Authors:

Pedro Vasques – ist1103665

Abdul Hannan Chowdhury – ist1112476

May 25, 2025

Table of Contents

Introduction	3
1. Kinematic Model – Denavit-Hartenberg Convention	4
1.1 Analysis of the Robot's Structure and Frame Assignment	4
1.2 Denavit-Hartenberg Parameters Table.....	5
2. Direct Kinematics Modeling and Simulation	6
2.1 Simulink Implementation Overview.....	7
2.2 Direct Kinematics Model Validation.....	8
2.3 Visualization of Reference Frames.....	8
3. Closed Form Solution for the Inverse Kinematics of the Robot – Kinematic Decoupling Approach	9
3.1 Simulink Implementation overview	10
3.2 Validation with the Direct Kinematics Model.....	10
4. Geometric Jacobian	11
4.1 Simulink Implementation Overview.....	12
4.2 Validation with Finite Differences	13
4.3 Singularity Detection.....	14
5 Closed Loop Inverse Kinematics.....	15
5.1 Simulink Implementation Overview.....	17
5.2 CLICK Model Validation.....	18
Conclusion.....	19
References	20

Introduction

This report presents the kinematic analysis of the Mako surgical manipulator developed by Stryker, as part of the coursework in Robotic Manipulation. The primary objective is to model and simulate the robot's motion using a systematic and symbolic approach, leveraging the MATLAB Symbolic Robotics Toolbox provided.

The analysis begins with the construction of a Denavit–Hartenberg (DH) kinematic model, developed from the robot's schematic. A custom joint diagram and corresponding DH parameter table are created to define the reference frames for each link.

Following this, a Simulink model for direct kinematics is implemented using the Toolbox. This model is validated through known configurations where the position and orientation of the end-effector are analytically predictable.

To address the inverse kinematics (IK) problem, a closed-form solution based on kinematic decoupling is derived and implemented in Simulink. The IK model is validated by comparing its output with that of the direct kinematics simulation.

The next stage involves computing the geometric Jacobian matrix. A Simulink model is built and validated by comparing its output with the numerical derivative of the direct kinematics. The analysis of kinematic singularities is also conducted, highlighting their impact on the rank and stability of the Jacobian.

Finally, the report presents a Closed-Loop Inverse Kinematics (CLIK) model, which utilizes the Jacobian and direct kinematics blocks. This approach provides a numerically robust alternative to analytical IK and is validated against the previously implemented models.

All models and simulations are developed using original MATLAB code, ensuring a complete and self-contained implementation of the Mako robot's kinematic behavior.

1. Kinematic Model – Denavit Hatenberg Convention

1.1. Analysis of the Robot's Structure and Frame Assignment

To model the Mako surgical robot using the Denavit–Hartenberg (DH) convention, the provided mechanical drawing (Figure 1) was analyzed to identify joint types, axis orientations, and inter-link relationships. The robot consists of six revolute joints (J1 to J6), forming a 6 degrees of freedom (DOF) manipulator. Each revolute joint allows rotation about its own axis, contributing one degree of freedom to the kinematic chain.

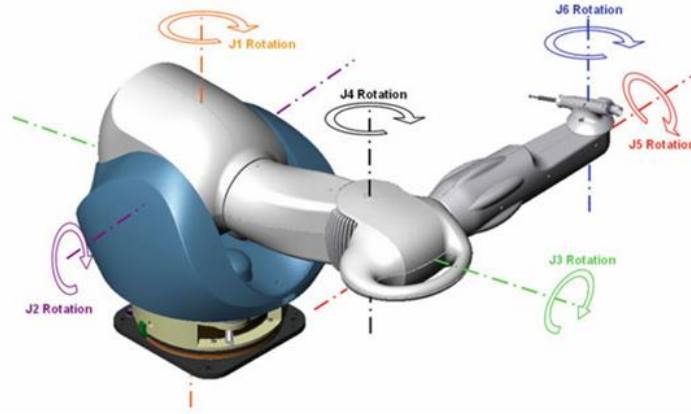


Figure 1: Mechanical drawing of Mako Surgical Robot

Observing Figure 1:

- Joints J1 and J2 make up the base and shoulder, allowing horizontal and vertical movement.
- Joint J3 provides an upward and downward motion, like an elbow.
- Joints J4 to J6 form a wrist that controls the orientation of the end-effector.

To build the kinematic model of the robot, the Denavit–Hartenberg (DH) Convention was used to define the relative position of consecutive links. This method standardizes the assignment of coordinate frames to each joint and link using four parameters (Figure 2): link length (a_i), link twist (α_i), link offset (d_i), and joint angle (θ_i) [1].

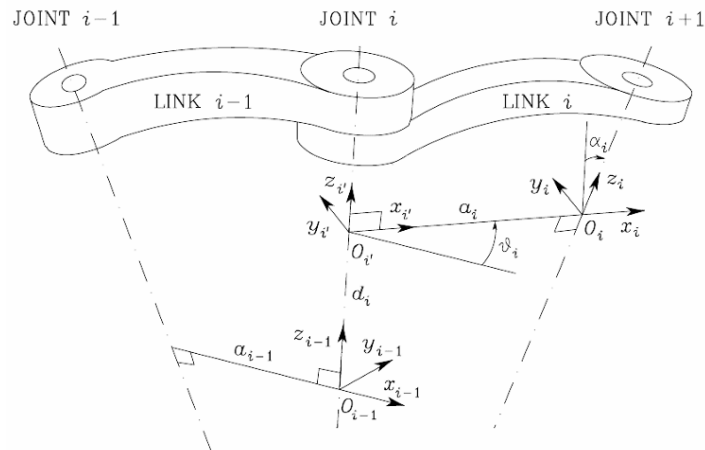


Figure 2: Denavit-Hatenberg (DH) parameters used to assign coordinate frames to mechanism joints

The procedure for defining the joint axis follows these steps:

1. Identify the joint axis z_i along the axis of joint $i + 1$.
2. Locate the origin O_i at the intersection of axis z_i with the common normal between axes z_{i-1} and z_i
3. Choose axis x_i along the common normal to axes z_{i-1} and z_i with direction from joint i to Joint $i + 1$
4. Choose axis y_i to complete a right-handed coordinate frame

Following this method, the Mako Surgical Robot joint axis are displayed in Figure 3.

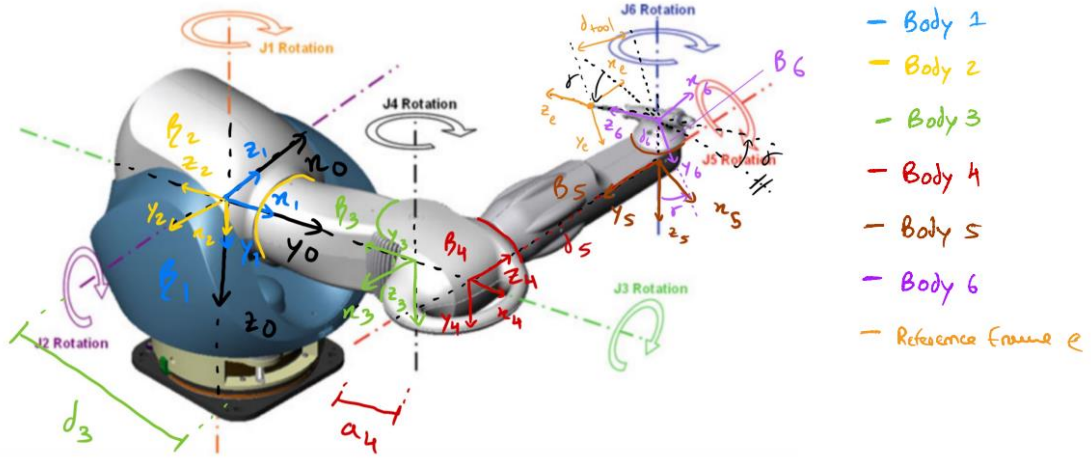


Figure 3: Mako Surgical Robot joint axis reference frames and DH parameters

The Robots fixed frame (x_0, y_0, z_0) is represented in black and it is located at the base of the robots' shoulder.

Each joint's rotation axis is clearly shown in Figure 3, and the coordinate frames follow the right-hand rule:

- Z-axes point along the joint rotation axes,
- X-axes are set perpendicular to both the previous and current Z-axes,
- Y-axes are defined as the cross product of Z and X: $Y_i = Z_i \times X_i$.

1.2. Denavit–Hartenberg Parameters Table

Table 1 presents the DH parameters for the Mako surgical robot. It includes six revolute joints and an additional frame at the end-effector. Each row corresponds to the respective link in the robot's kinematic chain.

Table 1: DH Parameters Table

Link\Parameter	ϑ_i (rad)	d_i (m)	a_i (m)	α_i (rad)	Offset (rad)
1	ϑ_1	0	0	$\pi/2$	$\pi/2$
2	ϑ_2	0	0	$-\pi/2$	$\pi/2$
3	ϑ_3	$-d_3$	0	$\pi/2$	$\pi/2$
4	ϑ_4	d_4	a_4	$\pi/2$	$-\pi/2$
5	ϑ_5	0	0	$-\pi/2$	0
6	ϑ_6	$-d_6$	0	$\pi/2 - \gamma$	$-\pi/2$
EE	0	d_{6p}	0	0	0

Following the D-H convention, the frame assigned to Link 6 (the last joint) does not coincide with the actual tool tip used in surgical procedures. Therefore, a separate end-effector frame (frame EE) is introduced at a distance d_{6p} along the Z_6 axis. This frame captures the precise position and orientation of the tool tip and is appended as an additional transformation in the model.

Throughout the report, to ensure realistic and consistent modeling, the constant DH parameters of the Mako Surgical Robot (Table 3) were assumed to be as follows:

$$\begin{aligned} d_3 &= 0.300 \text{ m} \\ d_5 &= 0.500 \text{ m} \\ d_6 &= 0.100 \text{ m} \\ d_{6p} &= 0.115 \text{ m} \\ a_4 &= 0.100 \text{ m} \\ \gamma_1 &= \pi/8 \text{ rad} \end{aligned}$$

It is important to note that all the link length (a_i) and link offset (d_i) values are constant given the fact that all the joints are revolute, so there is no variation in translation distance (only the joint angle).

To evaluate results using different values for the DH constants, these parameters can be manually modified within the `RIO.m` MATLAB script, which defines the robot's kinematic model and DH matrix parameters. The following analyses are automated through this function.

This sets the basis for building transformation matrices between consecutive link frames and enables computation of the robot's forward kinematics, where the overall pose of the robot's end effector relative to the base can be derived.

2. Direct Kinematics Modeling and Simulation

This section presents the implementation, validation and analysis of the direct kinematics of the Mako surgical robot. Direct kinematics refers to the calculation of the end-effector's position and orientation based on the joint parameters given (since all joints are revolute, $\vartheta_i = q_i$ in MATLAB scripts).

Using the DH convention, each link transformation is represented by a 4×4 homogeneous transformation matrix A_i^{i-1} which encodes both rotation and translation [1]:

$$A_i^{i-1} = \begin{bmatrix} \cos(\vartheta_i) & -\sin(\vartheta_i) \cos(\alpha_i) & \sin(\vartheta_i) \sin(\alpha_i) & \alpha_i \cos(\vartheta_i) \\ \sin(\vartheta_i) & \cos(\vartheta_i) \cos(\alpha_i) & -\cos(\vartheta_i) \sin(\alpha_i) & \alpha_i \sin(\vartheta_i) \\ 0 & \sin \alpha_i & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

To automate the construction of these transformation matrices, a MATLAB function named `DHTransf.m` was implemented. This function accepts a parameter vector $p = [d, \theta, a, \alpha, \text{offset}]$ and returns the symbolic homogeneous transformation matrix A for a single joint using the DH convention. It is worth noting that the key logic of the function includes identifying whether a joint is rotational or prismatic (by checking if ϑ is symbolic or not), and applying the appropriate joint offset either to d (for prismatic joints) or ϑ (for revolute joints). In this case, since all joints are revolute, the offset is always added to the ϑ parameter, shifting the joint angle accordingly in the transformation.

By sequentially multiplying these matrices for all n joints from the base to the end-effector, the full pose of the end-effector in the base frame is obtained [1]:

$$T_n^0 = A_0^1 \cdot A_1^2 \cdot \dots \cdot A_{n-1}^n$$

This cumulative transformation yields the position and orientation of the robot's end-effector with respect to the base frame. This process is automated with a MATLAB function named `Dkin.m`, that computes the homogenous transformation matrix using the DH parameters stored in the input matrix `Robot (RIO.m)`.

Using the provided Toolbox, the theoretical kinematic model was implemented and validated through a Simulink simulation environment.

2.1. Simulink Implementation Overview

A custom Simulink model for the direct kinematics of the robot, named `RIO_DKin_Simul.slx`, was developed (Figure 4) based on the DH parameters presented in Table 1 and derived in the first section.

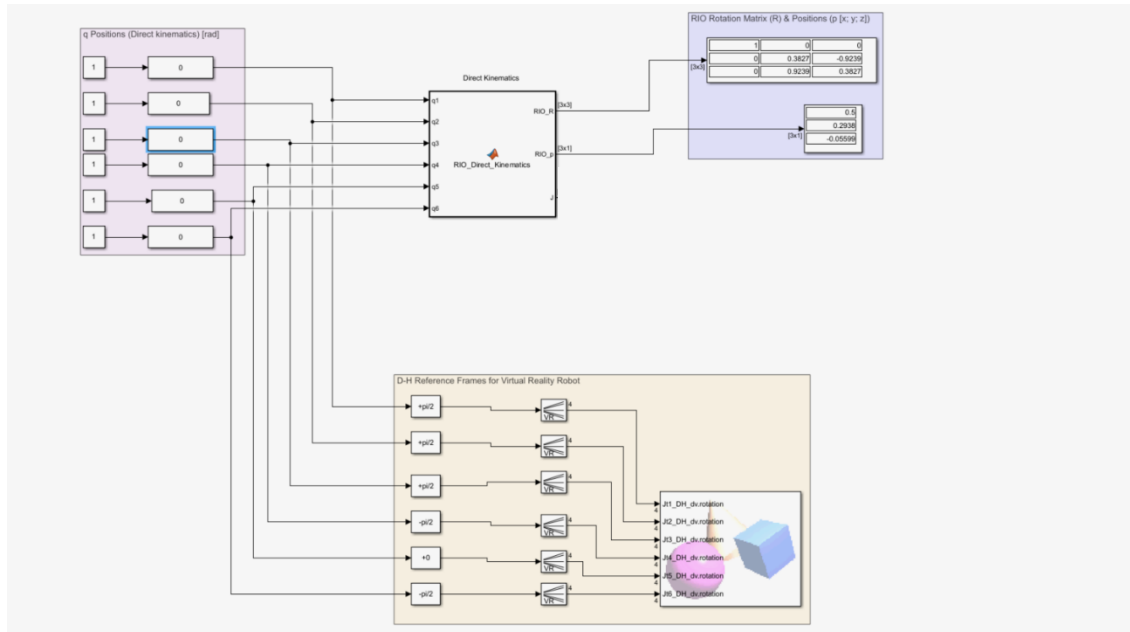


Figure 4: Direct kinematics Simulink model

The model receives joint angle inputs q_1 to q_6 (violet area in Figure 4), and computes the resulting transformation matrix, and outputs (blue area in Figure 4):

- The orientation of the end effector in reference to the robots' fixed frame (base) characterized by the rotation matrix `RIO_R` (3×3)
- The position of the end effector in reference to the robots' fixed frame (base), characterized e vector `RIO_p` (3×1)

The MATLAB function block `RIO_Direct_Kinematics` (Figure 4) is generated by the script `Dkin_Lib_example.m`. This script creates MATLAB and Simulink code for the robot model, specifically its direct kinematics, and encapsulates it within a reusable Simulink library block.

The block contains the implementation of the `Dkin.m` function on the DH parameters matrix defined in the `RIO.m` function. As mentioned before, the `Dkin.m` function outputs the transformation matrix from the robots' reference frame to the end effector. The orientation matrix (`RIO_R`) and the position component (`RIO_p`) are then extracted from the transformation matrix.

Additionally, each joint input is followed by an offset addition block (e.g., $+\pi/2$, $-\pi/2$, etc.) to align with the DH offset values (yellow area in Figure 4). These outputs are connected to virtual reality (VR) block for a 3-dimensional (3D) visualization of the robots' reference frames. This VR Sink block is also created in the `Dkin_Lib_example.m` script by means of the MATLAB function provided in the Toolbox `vrrobot.m`.

2.2. Direct Kinematics Model Validation

To validate the model for the direct kinematics of the robot, the robots all q positions were set to 0 rad (Figure 5), since the end-effectors position can be easily determined.

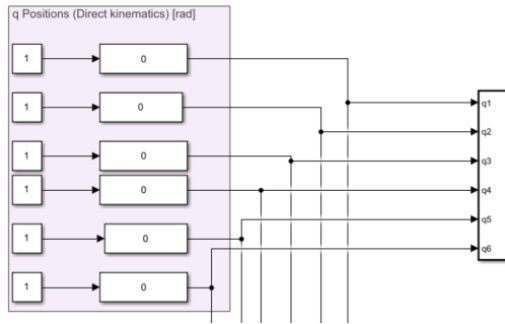


Figure 5: Simulink direct kinematics model q_i input values set to 0 rad

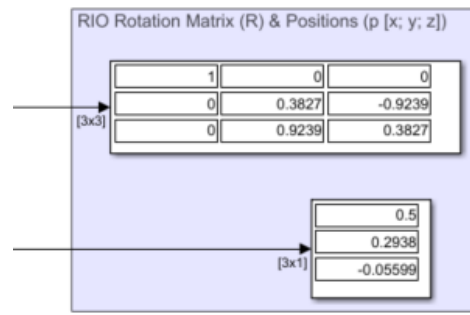


Figure 6: Simulink direct kinematics model outputs rotation matrix and position for all q_i set to 0 rad

The output obtained from these parameters is shown in Figure 6. When the robots q parameters are all set to 0 rad , it means that the joint angle (θ), more specifically the angle between the x_{i-1} and x_i axis for all links is null. This implies that the end-effector experiences no rotation about the reference frame's X-axis, which is confirmed by the first column of the orientation matrix: $[1 \ 0 \ 0]^T$. Additionally, the end effectors X-position should equal to the value of d_5 (which can be more easily visualized in Figure 7 and Figure 8), which is confirmed in Figure 6, as the position vectors X-coordinate is exactly 0.5 meters, as d_5 was defined in the first section. The rest of the output values also match the expected rotation and position values referent to the Y-axis and Z-axis frames.

2.3. Visualization of Reference Frames

The simulation includes VR-based visualization of the reference frames x_i, y_i, z_i for all joints $i = 0$ to 6, and $i = 7$ corresponding to the end-effector frame. Figures 7 and 8 illustrate the robots' joint axis positions for an input $q = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$ from two different viewpoints.

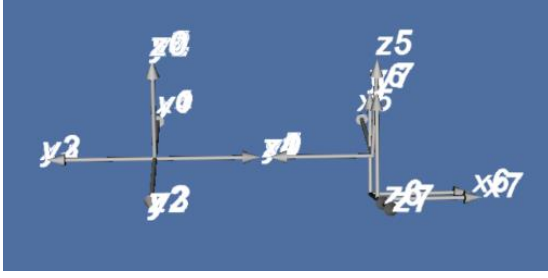


Figure 7: Robots reference frames with $q = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$ rad viewpoint 1

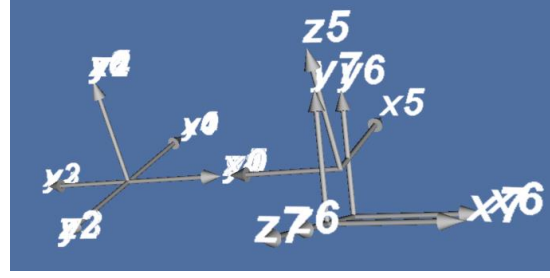


Figure 8: Robots reference frames with $q = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$ rad viewpoint 2

The position and orientation values of the end-effector obtained in the direct kinematic Simulink model seem to match the expected 3D visualization of the end-effector, especially regarding X-axis position and orientation values discussed in section 2.2..

This alignment across theoretical, simulation, and visual output validates the direct kinematics model of the Mako surgical robot. Such consistency indicates the model is accurate and well-suited for further studies in inverse kinematics and control.

3. Closed Form Solution for the Inverse Kinematics of the Robot – Kinematic Decoupling Approach

In the field of robot kinematics, inverse kinematics (IK) refers to the problem of determining the joint variables required to achieve a desired pose (position and orientation) of the end-effector. For a general 6-DOF manipulator, solving the inverse kinematics analytically can be complex or even intractable due to the nonlinear and coupled nature of the equations involved. However, for certain robot architectures—such as those with a spherical wrist (i.e., the last three joint axes intersecting at a single point)—a method known as kinematic decoupling allows for a closed-form solution.

Kinematic decoupling leverages the structural separation between the positioning and orientation tasks of the manipulator. This approach divides the inverse kinematics problem into two sub problems [2]:

- Position IK (first 3 joints): Solve for the first three joint angles (q_1, q_2, q_3) that position the wrist center (i.e., the intersection point of the last three joint axes) at the correct location in space.
- Orientation IK (Last 3 joints): Solve for the final three joint angles (q_4, q_5, q_6) that orient the end-effector as desired, assuming the wrist center has already been positioned correctly.

This decoupling is made possible by the RIO robots' kinematic structure, which includes a wrist with intersecting axes, making it suitable for such an analytical approach.

Let the desired end-effector pose be defined by the transformation matrix [1]:

$$T_d = \begin{bmatrix} R_d & p_d \\ 0 & 1 \end{bmatrix}$$

Where R_d is the desired rotation matrix and p_d is the desired position end effector vector. Knowing this, the wrist center p_w can be computed as:

$$p_w = p_d - R_d \cdot p_{tool}$$

Where p_{tool} is the offset vector from joint 6 to the tool tip, expressed in the end-effector frame. Once p_w is known, the problem is solved for q_1, q_2, q_3 using geometric or algebraic methods (e.g., cosine laws). After positioning the wrist, the orientation error between the wrist frame and the desired end-effector orientation and solve for q_1, q_2, q_3 by decomposing the resulting rotation matrix.

This process was implemented in a MATLAB function named `ClosedFormIK.m`, that takes the desired position and orientation as inputs and outputs the possible joint configurations.

3.1. Simulink Implementation overview

The implemented Simulink model for the closed form solution for the inverse kinematics of the robot is shown in Figure 9.

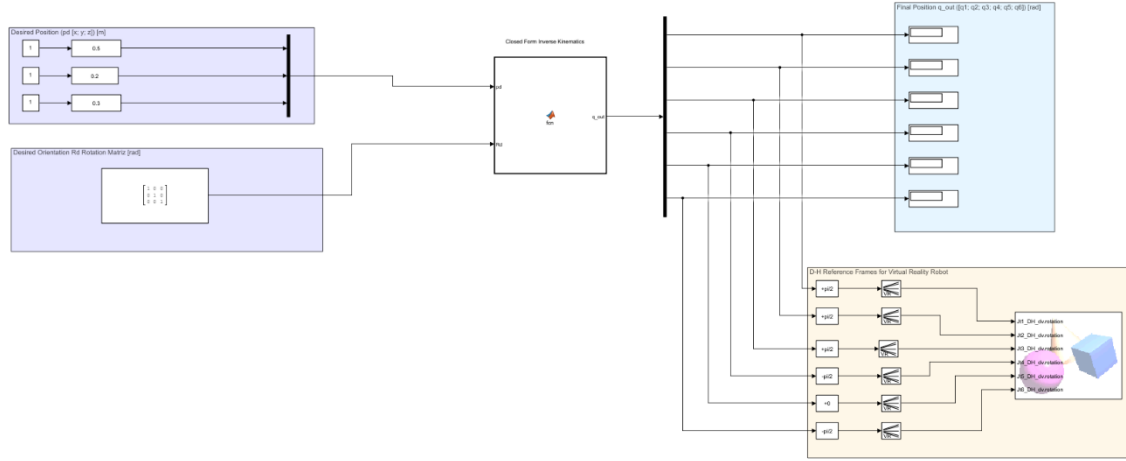


Figure 9: Simulink model for the closed form solution for the inverse kinematics

The model receives the Desired position p_d and orientation matrix R_d (dark blue areas in Figure 15) and computes the resulting joint configurations q_{out} that should characterize that desired pose.

This process is achieved by a MATLAB function block containing the `ClosedFormIK_Lib.m` function. This Simulink block is generated by the script `ClosedFormIK_Lib_example.m`. This script creates MATLAB and Simulink code for the robot model and encapsulates it within a reusable Simulink library block.

3.2. Validation with the Direct Kinematics Model

The resulting joint configurations can be used to compute the corresponding end-effector pose using direct kinematics. This pose is then compared against the desired one.

The results obtained for a $p_d = [0.2 \ 0.1 \ 0.4]^T$ meters and R_d set as a (3x3) identity matrix are displayed in Figure 10. When substituting the configuration joint values into the direct kinematics Simulink model, the output is noticeably different from the desired ones (Figure 11).

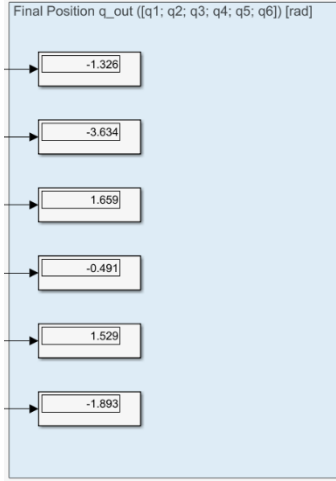


Figure 10: Closed form IK Simulink model final joint position output results

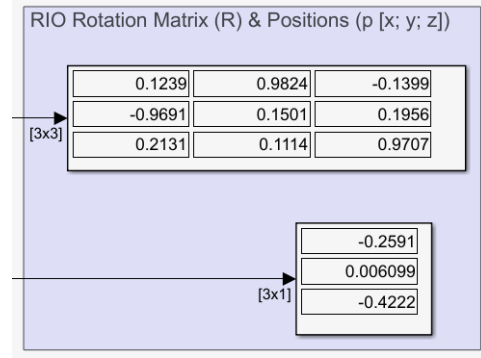


Figure 11: Simulink direct kinematics model outputs rotation matrix and position results

The fact that the value of q_2 (Figure 10) is lower than -2π radians is already a strong indication of an anomalous or physically implausible solution. These results are more easily addressed in the section ‘Validation with Direct Kinematics’ section inside `ClosedFormIK_Lib_example.m` the MATLAB file.

These issues are probably derived from several common implementation challenges, such as an incorrect identification of the wrist center, improper treatment of the end-effector offset relative to the final joint, misinterpretation of joint coordinate frames, or the use of ambiguous inverse trigonometric solutions without appropriate quadrant handling. Additionally, failure to constrain joint limits or resolve multiple solution branches can lead to results that are physically implausible or outside the robot's reachable workspace.

However, it is important to highlight that these shortcomings stem primarily from implementation errors rather than limitations of the closed-form method itself. When correctly formulated, closed-form solutions provide highly efficient, real-time inverse kinematics computation without requiring iterative methods or approximations. They offer clear analytical insight into the robots' geometry and multiple valid configurations, making them especially valuable in applications requiring fast, deterministic motion planning.

4. Geometric Jacobian

This section presents the construction, validation, and analysis of the geometric Jacobian for the Mako surgical robot. The Jacobian is a key matrix in robotics that relates the joint velocities $\dot{q}$ to the linear and angular velocity of the end-effector [1]:

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix} = J(q) \cdot \dot{q} \quad (4.1)$$

where $\xi \in \mathbb{R}^6$ is the spatial velocity (also known as the twist) of the end-effector composed of:

- $v \in \mathbb{R}^3$: linear velocity of the end-effector origin
- $\omega \in \mathbb{R}^3$: angular velocity to the end-effector frame

Using the homogeneous transformation matrices derived from the DH convention, the Jacobian was computed symbolically in MATLAB. For each joint i , two key vectors were extracted:

- o_i : the origin of frame i

- z_i : the unit vector along the joint's axis of rotation

For revolute joints, the contribution of joint i to the end-effector motion is calculated as [1]:

$$J_i = \begin{bmatrix} z_{i-1} \times (o_n - o_{i-1}) \\ z_{i-1} \end{bmatrix}$$

with:

$$\begin{aligned} z_{i-1} &= R_1^0(q_1) \dots R_{i-1}^{i-2}(q_{i-1}) z_0, & z_0 &= [0 \ 0 \ 1]^T \\ \widetilde{o}_e &= A_1^0(q_1) \dots A_n^{n-1}(q_n) \widetilde{o}_0, & \widetilde{o}_0 &= [0 \ 0 \ 0 \ 1]^T \\ \widetilde{o}_{i-1} &= A_1^0(q_1) \dots A_{i-1}^{i-2}(q_{i-1}) \widetilde{o}_0 \end{aligned}$$

where all vectors are expressed in the base frame. These columns are concatenated to form the full $6 \times n$ Jacobian matrix, where n is the total number of joints ($n = 6$ in the present study). This process was implemented and automated in the MATLAB function `Dkin.m` (where the transformation matrix is also computed) by walking through the chain of transformations from the base to the end-effector. This means that `Dkin.m` now receives as inputs the DH matrix parameters and outputs both the transformation matrix and the geometric Jacobian of the robot

4.1. Simulink Implementation Overview

The implemented Simulink model is similar to the one described in Section 2.1, as it was developed on top of the same simulation file `RIO_DKin_Simul.slx`.

Since the `Dkin.m` was updated to compute also geometric Jacobian matrix of the robot, the only change in the additional output J correspondent to the Jacobian in the MATLAB function block `RIO_Direct_Kinematics` (highlighted in red in Figure 12).

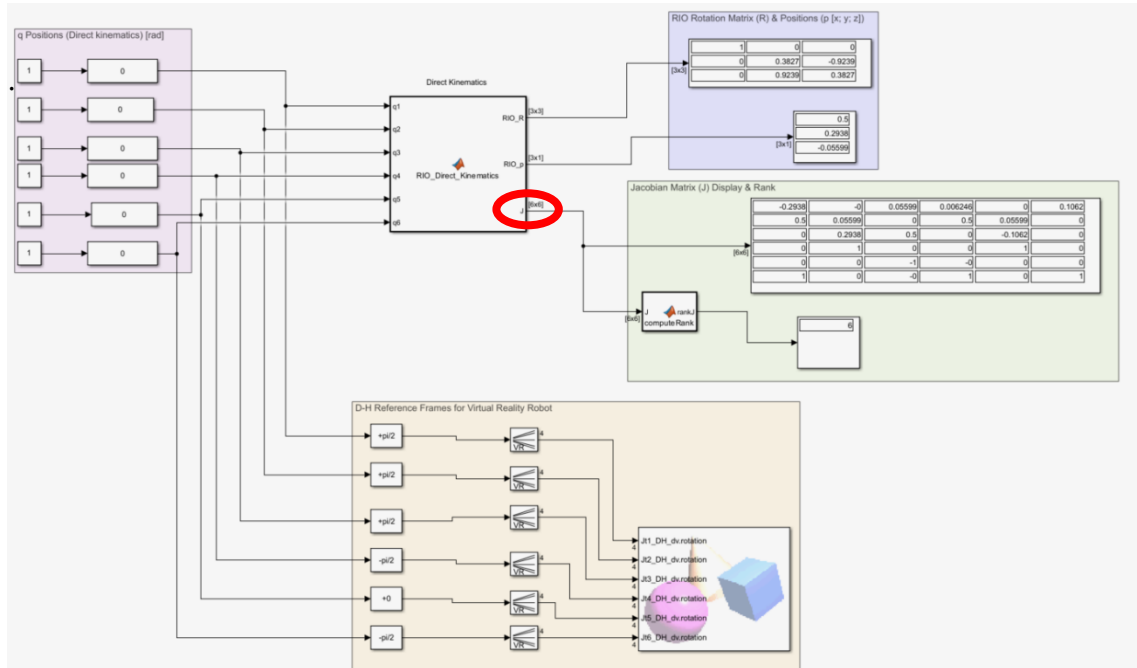


Figure 12: Simulink model for the direct kinematics with Jacobian computation

The output J is connected to a display block, that shows the Jacobian matrix for the different joint configurations. It is also connected to an additional MATLAB function block that computes the rank of matrix J , which is likewise shown in a display block (green area in Figure 12).

The Jacobian matrix results for an input $q = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$ are displayed in Figure 13.

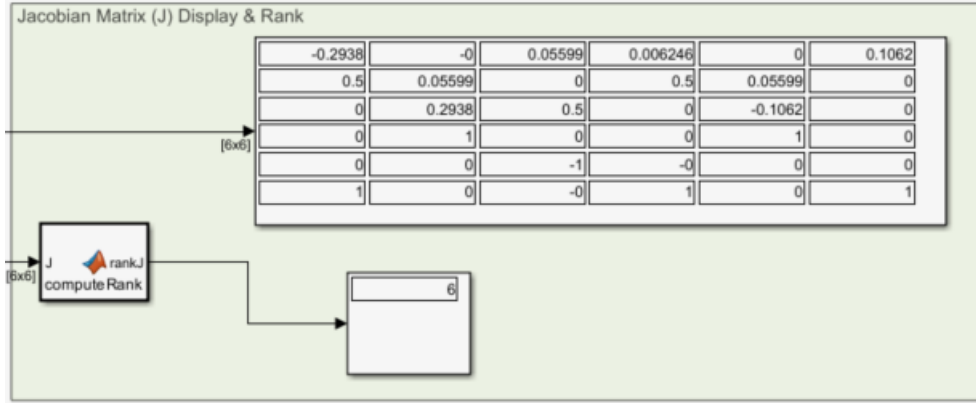


Figure 13: Robots geometric Jacobian display and rank for an input $q = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$

The results initially appear to be consistent and reasonable, especially since the matrix rank is equal to 6 at this configuration, which suggests that the robot is in a non-singular posture and retains full mobility in all six degrees of freedom as expected for this joint configuration. However, the specific values of the Jacobian matrix entries require further validation to ensure their numerical accuracy.

4.2. Validation with Finite Differences

To validate the correctness of the Jacobian, a numerical differentiation approach was applied. Since the geometric Jacobian must satisfy the relationship described in equation (3), to verify this mapping, for a given joint configuration, each joint variable q_i is perturbed by a small amount ε , and the corresponding change in pose is computed using forward kinematics (`Dkin.m`). The Jacobian column corresponding to q_i is approximated using central finite differences:

- Linear velocity:

$$v_i \approx \frac{p(q + \varepsilon e_i) - p(q - \varepsilon e_i)}{2\varepsilon}$$

- Angular velocity:

$$w_i \approx \text{vee} \left(\frac{R(q + \varepsilon e_i) - R(q - \varepsilon e_i)}{2\varepsilon} R(q)^T \right)$$

where e_i is the standard basis vector for joint i , and $\text{vee}(\cdot)$ extracts the angular velocity vector from the skew-symmetric matrix.

This yields a numerical Jacobian $J_{num} \in \mathbb{R}^6$, which is then compared with the symbolic Jacobian evaluated at the same configuration by subtracting one from the other and analyzing the resulting difference. Additional matrix summary metrics, such as the rank and condition number, are also computed to further support the validation process. All this process is implemented on a MATLAB script named `JacobianND_Singularities_Test.m`.

Using the same input values as in the Simulink implementation where q is set to $[0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$, and using a small perturbation $\varepsilon = 1 \times 10^{-6}$, the results present in Figures 14 and 15 were obtained.

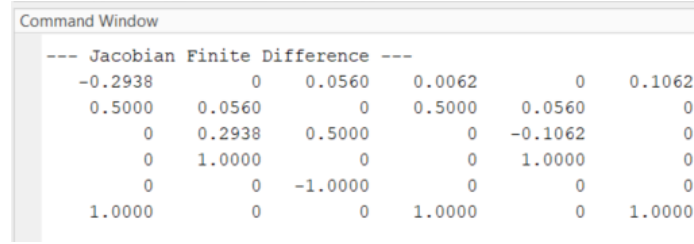


Figure 14: Finite differences Jacobian output for $q = [0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$, `JacobianND_Singularities_Test.m`

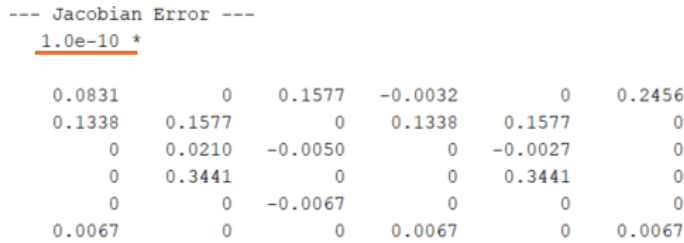


Figure 15: Jacobian entries error computation output for $q = [0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$, `JacobianND_Singularities_Test.m`

When comparing the Jacobian obtained with finite differences (Figure 14) with computed through the simulation (Figure 13), the results are virtually identical. Moreover, the maximum Jacobian error in any individual matrix entry is in the order of 1×10^{-11} (Figure 15).

This validation approach ensures that the Jacobian not only reflects the kinematic structure of the manipulator but also that its differential mapping between joint space and task space is functionally accurate. It provides confidence that the model can be reliably used for velocity control, singularity analysis, and Closed-Loop Inverse Kinematics (CLIK) applications.

4.3. Singularity Detection

An analysis of kinematic singularities was performed by monitoring the rank of the Jacobian matrix across various joint configurations. A configuration is considered singular when the Jacobian loses rank (< 6), that is, when some rows or columns become linearly dependent. In such cases, the robot loses the ability to move in certain directions or requires infinitely large joint velocities to achieve specific Cartesian motions.

To facilitate the identification and interpretation of these singularities, they were also classified into arm and wrist singularities. Arm singularities typically occur when the first three joints (responsible for positioning the end-effector) align in such a way that reduces spatial reach, while wrist singularities arise from the alignment of the last three joints (responsible for orientation), often causing a loss of rotational degrees of freedom. This classification simplifies the problem and helps isolate the source of the singularity.

This process is automated in MATLAB named `JacobianND_Singularities_Test.m`. High computational cost and long execution time were experienced when attempting to solve for exact singularities expressions analytically. To address this, it was adopted a brute-force random search over a discrete joint grid. Although coarse and structured, this approach approximates a random sampling method, increasing the chances of detecting near-singular configurations without relying on analytical solutions.

A selection of the singularity results obtained is shown in Figure 16.

Command Window									
Singular configurations detected:									
q1	q2	q3	q4	q5	q6	Rank	MinSingVal	CondNum	Type
-3.14	-3.14	-3.14	-3.14	-1.57	-3.14	5	2.48e-17	6.45e+16	General
-3.14	-3.14	-3.14	-3.14	-1.57	-1.57	5	0.00e+00	Inf	General
-3.14	-3.14	-3.14	-3.14	-1.57	0.00	5	2.48e-17	6.45e+16	General
-3.14	-3.14	-3.14	-3.14	-1.57	1.57	5	0.00e+00	Inf	General
-3.14	-3.14	-3.14	-3.14	-1.57	3.14	5	2.48e-17	6.45e+16	General
-3.14	-3.14	-3.14	-3.14	1.57	-3.14	5	5.36e-18	2.96e+17	General

Figure 16: Singularities detection output snippet example, `JacobianND_Singularities_Test.m`

Another obvious singularity is when the Mako robots' arm is fully extended with the joint configuration $q = [0 - \pi/2 \ 0 \ 0 \ \pi/2 \ 0]^T \text{ rad}$. When substituting any of the obtained singularity solutions into the Simulink model, the displayed rank is consistently lower than 6, confirming the expected behavior (see Figure 17).

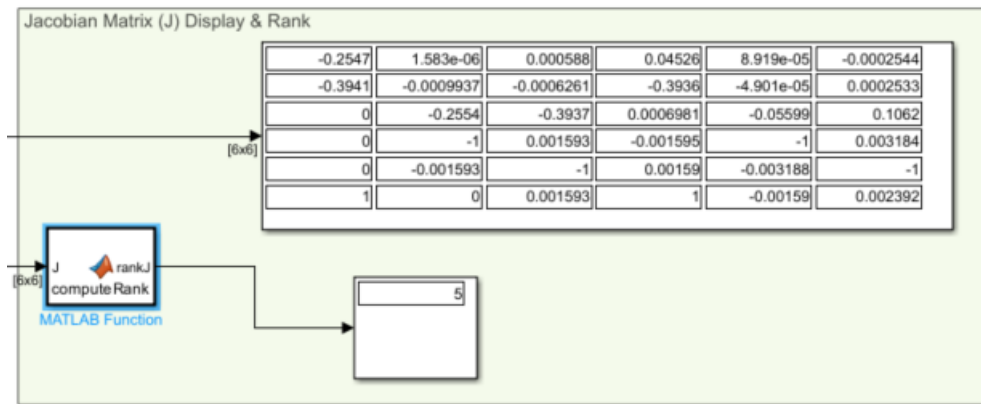


Figure 17: Robots geometric Jacobian display and rank for an input $q = [-3.14 \ -3.14 \ -3.14 \ -3.14 \ -1.57 \ -1.57]^T \text{ rad}$

It is important to note that when computing the rank in MATLAB, a tolerance threshold must be applied; otherwise, in this particular case, the rank will almost always be reported as 6, even near singular configurations.

5 Closed Loop Inverse Kinematics

In robotic manipulation, inverse kinematics (IK) involves computing the joint variables that produce a desired pose (position and orientation) of the end-effector. Unlike analytical IK, which

is often complex or even infeasible for general 6-DOF arms, Closed-Loop Inverse Kinematics (CLIK) offers a numerically stable, iterative alternative.

CLIK methods are based on differential kinematics: they compute joint velocities from Cartesian errors and integrate them over time to update joint positions. The foundation is the geometric Jacobian which relates joint velocities to end-effector linear and angular velocities (Equation 4.1). Given a desired pose (p_d, R_d) , and the current pose (p, R) , CLIK computes the Cartesian error [1]:

- Position error:

$$e_p = p_d - p$$

- Orientation error:

$$e_o = \frac{1}{2} \text{vec}(R_d R^T - R R_d^T)$$

In practice, the angular part is computed from the skew-symmetric difference of rotation matrixes:

$$e_o = \begin{bmatrix} R_{e32} & R_{e23} \\ R_{e13} & R_{e31} \\ R_{e21} & R_{e12} \end{bmatrix}$$

The combined special vector $e \in \mathbb{R}^6$ is:

$$e = \begin{bmatrix} e_p \\ e_o \end{bmatrix}$$

To reduce this error a proportional feedback control law is applied:

$$v_e = K_p e$$

where v_e is the control twist (desired end-effector velocity) and K_p the proportional gain matrix.

Using this control input, joint velocities are computed via the pseudo-inverse of the Jacobian:

$$\dot{q} = J^\dagger(q) v_e$$

And joint values are iteratively updated using:

$$q_{k+1} = q_k + \dot{q}_k \cdot \Delta t$$

This feedback loop runs until the error norm $\|e\|$ falls below a specified tolerance.

The CLIK algorithm has been implemented and automated in a MATLAB function called `CLIK.m` using the symbolic models previously developed for the forward kinematics (Section 2), and the geometric Jacobian (Section 4). This function takes as inputs:

- Initial robot configuration q_0
- Desired position p_d
- Desired orientation matrix R_d
- Time interval dt
- Tolerance tol

- Maximum number of iterations max_iter

And outputs the joint configuration that minimizes the Cartesian error, in other words, the joint configuration that better approximates the desired pose.

5.1. Simulink Implementation Overview

A custom Simulink model CLICK of the robot, named `CLICK_RIO_Simul.slx`, was developed (Figure 18).

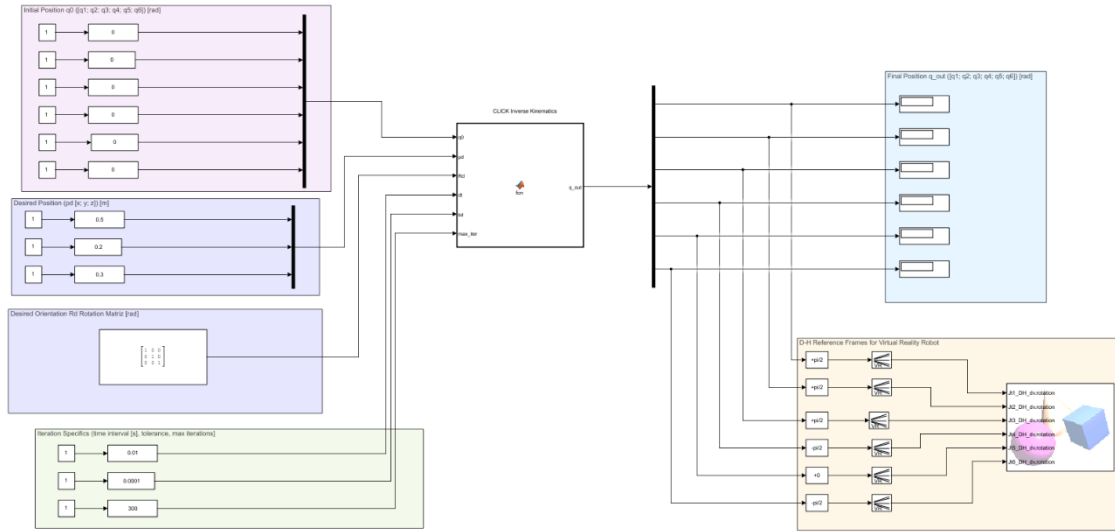


Figure 18: Simulink CLICK model

The model takes as inputs:

- Initial robot configuration q_0 (magenta area in Figure 18)
- Desired position p_d and orientation matrix R_d (dark blue areas in Figure 18)
- Iteration specifics: time interval dt , tolerance tol and maximum number of iterations max_iter (green area in Figure 18)

which are computed by a MATLAB function block containing the `CLICK.m` function. This Simulink block is generated by the script `CLICK_Lib_example.m`. This script creates MATLAB and Simulink code for the robot model, specifically its CLICK model, and encapsulates it within a reusable Simulink library block.

The model then outputs the joint configuration (q_{out}) that better approximates the desired pose (light blue are in Figure 18).

5.2. CLICK Model Validation

Given the results obtained in Section 3 were clearly not satisfactory, it was decided to validate the model using direct kinematics. The resulting joint configurations can be used to compute the corresponding end-effector pose using direct kinematics. This pose is then compared against the desired one, confirming that the spatial error approaches zero as the algorithm converges.

Figure 19 displays the joint configuration output for the given specifications:

- $q_0 = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T \text{ rad}$
- $p_d = [0.5 \ 0.2 \ 0.3]^T \text{ m}$
- $R_d = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ rad}$
- $\Delta t = 0.01 \text{ s}$
- $tol = 1 \times 10^{-4}$
- $max_{iter} = 300$

When substituting the configuration joint values into the direct kinematics Simulink model, the output closely approximates the desired position (Figure 20).

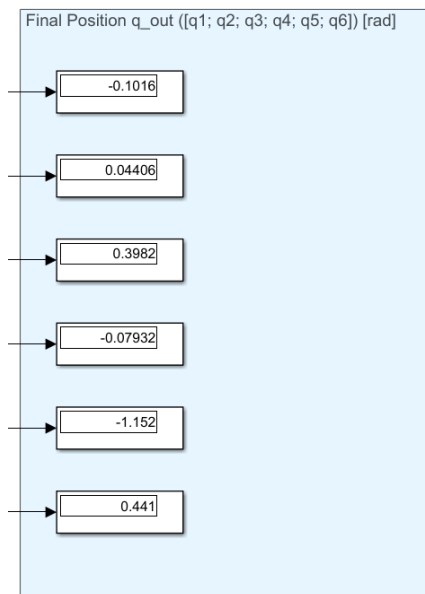


Figure 15: CLICK Simulink model final joint position output results

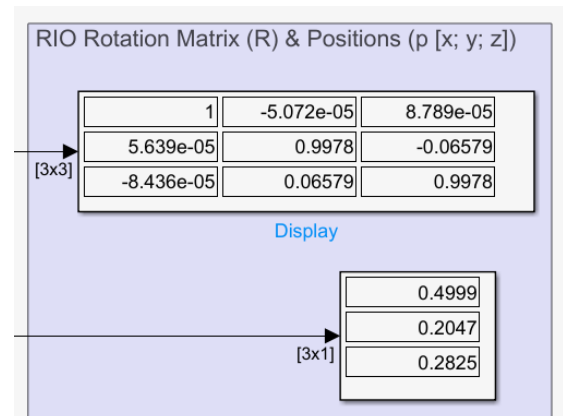


Figure 20: Simulink direct kinematics model outputs rotation matrix and position results

The results demonstrate the validity of the CLICK model implementation. These results are more easily verified through the ‘Validation with the Direct Kinematics’ section of the MATLAB script named `CLICK_Lib_example.m` which compares directly both values in the same manner.

It is important to note that multiple joint configurations can lead to the same desired end-effector pose, highlighting the inherent redundancy and multiplicity of solutions in the CLICK model. Consequently, the CLICK model cannot be verified in reverse—that is, by checking the results in the Direct Kinematics model, substituting them as inputs into the CLICK model, and expecting the same joint configuration as output.

Conclusion

This project successfully addressed the kinematic modeling and simulation of the Mako surgical robot using a structured approach grounded in the Denavit–Hartenberg (DH) convention. Beginning with an analysis of the robot’s geometry, a complete DH parameter table and frame assignment were established, forming the foundation for the robot’s forward kinematics.

Using the provided Symbolic Robotics Toolbox, a Simulink model for direct kinematics was implemented and validated through known configurations, with the outputs aligning closely with theoretical expectations and visual representations. A closed-form inverse kinematics (IK) solution, enabled by the robot’s spherical wrist structure, was derived using a decoupled approach. Although the implementation faced challenges in handling joint offsets and solution branches, the method itself remains valid and efficient for robots with similar architecture.

The geometric Jacobian was computed symbolically and validated through numerical differentiation using finite differences. The results showed negligible errors, confirming the accuracy of the symbolic model. Singularities were systematically detected by analyzing the rank of the Jacobian across sampled joint configurations and were further classified into arm and wrist singularities. Due to the computational difficulty of deriving exact singular conditions analytically, a structured brute-force sampling method was employed, effectively identifying near-singular configurations.

Finally, a Closed-Loop Inverse Kinematics (CLIK) model was developed using the Jacobian and forward kinematics blocks. This iterative method demonstrated reliable convergence to desired poses, with results validated against the direct kinematics outputs. The implementation highlights the effectiveness of numerically stable IK solutions, especially in scenarios where analytical approaches are limited or fail.

In summary, all assigned tasks were addressed through original MATLAB code and simulations, providing a comprehensive kinematic analysis of the Mako surgical manipulator, and laying the groundwork for future dynamic studies or control implementations.

References

- [1] J. M. M. Martins, *Kinematics and Trajectory Planning*. [PDF], Available on Fenix course page.
- [2] A. S. Carvalho and J. M. M. Martins, *Inverse Kinematics of Robotic Manipulators*. [PDF], Available on Fenix course page.